

# ASRR: Offline Action Sequence Residual Refinement for Robot Policy Adaptation

Anonymous Authors

**Abstract**—Robots deployed with limited local computation and intermittent remote support need efficient ways to improve an existing policy. Such a policy may already capture task intent and propose reasonable actions, yet local inaccuracies can still cause failures. Further fine-tuning or reinforcement learning can require costly optimization or additional interaction. This motivates us to investigate whether a useful action proposal can itself serve as the target of adaptation. We introduce *Action Sequence Residual Refinement* (ASRR) to test this hypothesis. ASRR keeps the base policy frozen and trains a lightweight refiner offline from cached policy outputs and existing demonstrations. The refiner predicts a small residual over each proposal, optionally using policy-side context. On  $\pi_{0.5}$ , context-conditioned ASRR improves average success across the four LIBERO suites by 14.4 percentage points. Across checkpoints from the same fine-tuning run, ASRR raises average success from 62.0% to 80.9% at 5k and from 91.3% to 95.9% at 30k, supporting the targeted competent-but-imperfect regime. In adjacent-checkpoint comparisons, recorded cache extraction and refiner fitting use 25–63% fewer GPU-hours than continued LoRA optimization. Across three real-robot manipulation tasks, the same interface improves aggregate success by 6.7 percentage points. These results establish predicted action sequences as editable adaptation interfaces and provide a lightweight, offline approach to improving already capable robot policies.

## I. INTRODUCTION

Robot learning has progressed from visuomotor policies that map observations to low-level control toward policies that predict temporally structured future actions. Action Chunking with Transformers (ACT) predicts coherent action chunks, Behavior Transformers model multimodal behavior over action sequences, and Diffusion Policy (DP) generates continuous trajectories through conditional denoising [1]–[3]. More recently, vision-language-action (VLA) systems have coupled visual and language representations to robot control at increasing scale [4]–[11]. Despite their architectural differences, these systems ultimately require an action-generation mechanism, including autoregressive token prediction, continuous chunk regression, diffusion, or flow matching, to produce executable control. Their predicted actions therefore form a common interface between policy computation and physical execution.

Practical deployment can constrain the resources available for further adaptation. Subterranean exploration faces on-board computation and communication limits [12], while untethered underwater operations contend with low-bandwidth, intermittent communication [13]. These examples motivate a broader setting: a robot can execute an existing policy, but has a limited budget for improving it. In manipulation, a policy may approach the intended object or placement region yet fail because of local pose, motion, or timing errors [14],

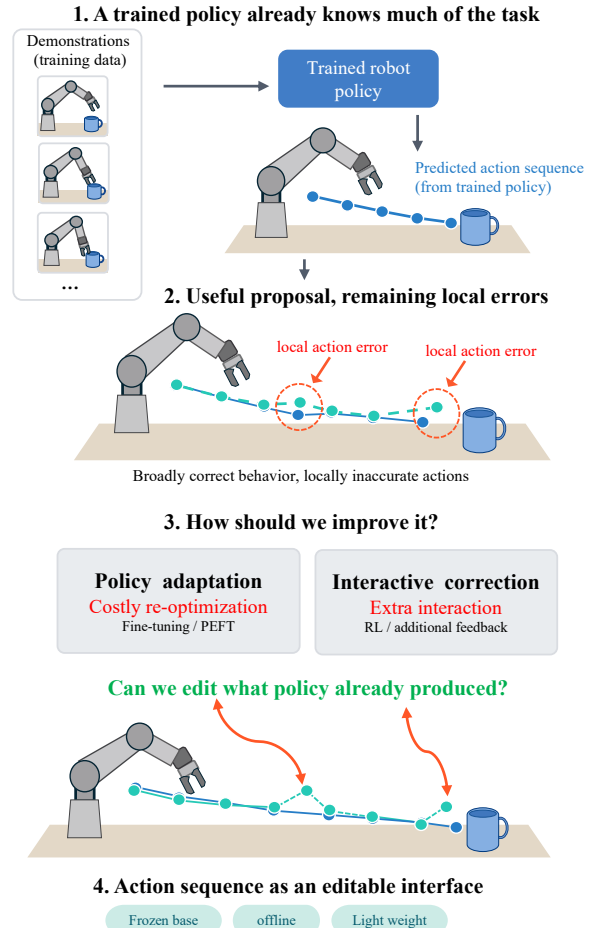


Fig. 1. A competent but imperfect policy can produce a useful action proposal. ASRR edits the output sequence before execution, leaving the policy unchanged.

[15]. We refer to this practically important setting as the *competent-but-imperfect* regime: the base policy already provides meaningful task behavior, while local action errors still limit execution reliability.

Existing adaptation strategies can address such failures, but often at nontrivial cost. Full fine-tuning and parameter-efficient methods such as adapters or LoRA continue to optimize policy parameters or internal representations, requiring another round of model training [11], [16], [17]. Dataset aggregation collects learner-induced states, while residual reinforcement learning and reinforced VLA fine-tuning may introduce additional interaction, reward or value optimization, and rollout collection [15], [18]–[21]. These

interventions are valuable when a new skill, representation, or exploration strategy must be learned. When the base already produces a useful action proposal, however, repeatedly updating the policy or collecting additional interaction can be unnecessarily costly for correcting localized action errors.

This motivates a different adaptation target: the action proposal itself. Predicted chunks and trajectories are normally treated as commands to execute, but they also encode the intended near-future behavior of the policy. If the direction of that behavior is meaningful but locally inaccurate, correction need not relearn the perception, grounding, and policy computation that produced it. The predicted action sequence can instead serve as an editable intermediate representation, providing a compact surface for post-hoc adaptation.

We introduce *Action Sequence Residual Refinement* (ASRR),<sup>1</sup> a lightweight offline method for editing action proposals from a frozen robot policy (Fig. 1). ASRR trains a small *Action Sequence Refiner* from cached base-policy outputs and existing demonstrations. The refiner can operate on the proposal alone or use policy-side context when additional information is needed to disambiguate it, and predicts a conservative residual before the native execution protocol is applied. Because the base policy remains frozen, refiner training is performed offline on cached proposals, without backpropagation through the large policy or reward-driven interaction.

Our experiments progress from conventional action-sequence policies to large VLAs. On ACT, action-only refinement improves insertion success by 21.7 percentage points; DP shows the value of observation and trajectory-mode context. On  $\pi_{0.5}$ , ASRR improves all four LIBERO suites, averaging 14.4 points with suite-specific refiners and 13.7 points with one shared refiner. The same-run checkpoint study shows larger absolute gains at earlier checkpoints. Across three adjacent-checkpoint comparisons, recorded cache extraction and refiner fitting require 25–63% fewer GPU-hours than continued LoRA optimization (Table II). ASRR also yields positive mean gains on the strong OpenVLA-OFT base. With a Piper arm, it improves all three real-robot tasks and raises aggregate success by 6.7 percentage points.

Our contributions are threefold:

- **Editable action proposals.** We establish predicted action sequences as editable interfaces, enabling direct action-level correction instead of further policy optimization.
- **Lightweight offline refinement.** We introduce ASRR, which freezes the base policy and learns compact residual refiners offline from cached proposals and existing demonstrations, without reward-driven interaction.
- **Broad simulation and real-robot validation.** Across action-sequence policies and modern VLAs, ASRR achieves substantial gains in simulation and on real robots with a small trainable refiner.

## II. RELATED WORK

*a) Action-sequence policies and VLA action generation:* Action-sequence prediction is a mature paradigm for robot control. ACT showed that action chunking with temporal aggregation can support precise manipulation, Behavior Transformers modeled multimodal behavior through sequence prediction, and DP introduced generative trajectory modeling for visuomotor control [1]–[3]. VLA policies combine large-scale visual and language representations with action generation: RT-1, RT-2, and OpenVLA use tokenized or autoregressive action prediction [4]–[6], while  $\pi_0$  and  $\pi_{0.5}$  generate continuous action chunks through flow-based models [9], [10]. Octo and SmolVLA further study generalist and lightweight robot policies [7], [8].

*b) Policy adaptation and efficient fine-tuning:* Adapting a pretrained policy is commonly performed by updating model parameters. Full fine-tuning offers flexibility but becomes increasingly expensive as model size grows. Parameter-efficient methods reduce this cost: adapters insert small trainable modules into an otherwise frozen network [16], while LoRA represents task-specific updates through low-rank modifications of existing weights [17]. OpenVLA-OFT further studies efficient VLA adaptation together with continuous action prediction and parallel decoding [11]. Beyond supervised fine-tuning, ConRFT and CO-RFT introduce reinforced or offline-RL objectives for VLA adaptation [20], [21]. Despite differences in supervision and parameterization, these approaches modify the policy itself. Our setting instead keeps the selected policy checkpoint fixed and applies correction after action generation.

*c) Residual and corrective control:* Residual Policy Learning and Residual RL established the idea of improving an existing controller with an additional learned correction, typically optimized through environment interaction [15], [19]. ResiP extends this principle to chunked behavior-cloning policies by learning a closed-loop residual controller for precise assembly [22]. Recent work further applies corrective control to large and generative robot policies: object-centric residual RL learns a simulation-trained residual over a frozen VLA [23], while FlowCorrect adapts flow-based policies from sparse human corrections at deployment time [24]. ASRR shares the goal of preserving useful base behavior, but uses a different correction regime: it learns offline from existing demonstrations and directly refines the predicted action sequence without online reinforcement learning or deployment-time corrective feedback.

## III. ACTION SEQUENCE RESIDUAL REFINEMENT

ASRR follows a simple principle: once a robot policy produces a meaningful future action sequence, adaptation can act directly on that proposal while the generating policy remains fixed. Given a frozen base policy, ASRR takes the proposed sequence as its primary input, optionally augments it with context already exposed by the policy, and predicts a lightweight residual in the same action space. The refined proposal is then passed to the policy’s original temporal execution mechanism. Training uses cached base proposals

<sup>1</sup>Project page: <https://anonymous-0525.github.io/ASRR/>

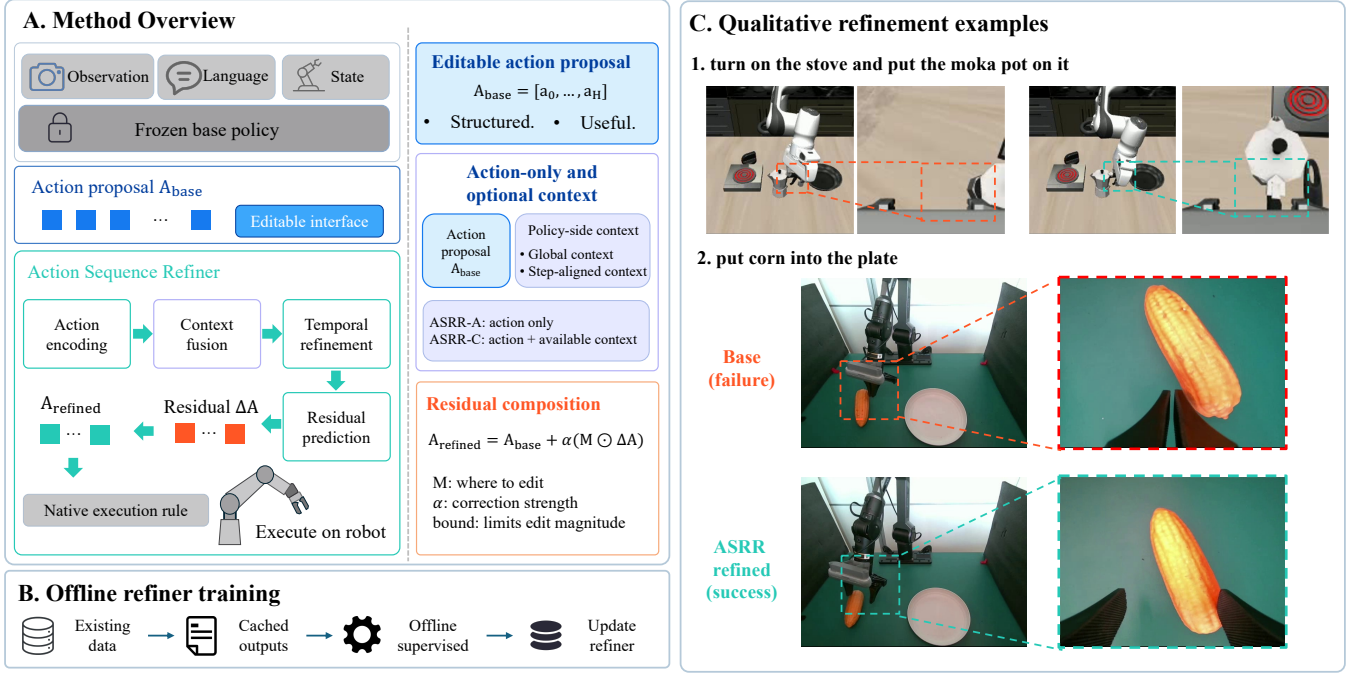


Fig. 2. ASRR refines the action sequence proposed by a frozen policy. Demonstrations supervise offline sequence editing; deployment applies the refined proposal through the base policy’s native execution rule.

paired with demonstration actions, so only the small refiner is optimized (Fig. 2).

#### A. Editing a Frozen Action Proposal

Let  $f_{\text{base}}$  denote a frozen action-generating policy. At control time  $t$ , it maps observation  $o_t$  and optional task or language condition  $c_t$  to

$$A_{b,t} = f_{\text{base}}(o_t, c_t) = [a_{b,t}^0, \dots, a_{b,t}^{H_p-1}] \in \mathbb{R}^{H_p \times d_a}, \quad (1)$$

where  $H_p$  is the prediction horizon and  $d_a$  is the dimension of the policy-native action representation. ASRR treats  $A_{b,t}$  as an editable proposal. In its minimal form, an Action Sequence Refiner maps this proposal to a residual  $\Delta A_t = R_{\theta}(A_{b,t})$ , which is composed as

$$\tilde{A}_t = A_{b,t} + \alpha(M \odot \Delta A_t), \quad (2)$$

where  $\alpha$  controls correction strength and  $M$  masks action coordinates or timesteps. The final residual projection is initialized to zero, so the composed controller initially reproduces the frozen base exactly.

The residual formulation is designed for proposals that already contain useful task structure, so the refiner only needs to make local corrections. When the proposal is already close to successful execution, large residuals are unnecessary and potentially harmful. ASRR therefore limits its intervention through zero initialization, masks, bounds, and residual scale.

#### B. Action-Only and Context-Conditioned Refinement

The minimal variant, which we denote ASRR-A, operates only on the predicted action sequence. It directly tests whether the proposal contains enough information for correction and avoids introducing a separate perception pathway.

For some policies, however, similar proposed trajectories may require different edits depending on robot state, task, or latent generation mode. We therefore define ASRR-C, which conditions refinement on the policy-side context available in the chosen configuration.

The refiner represents each proposed action as an editable token,

$$x_h = \phi_a(a_{b,t}^h) + p_h, \quad h = 0, \dots, H_p - 1, \quad (3)$$

where  $\phi_a$  is a lightweight action encoder and  $p_h$  is a temporal positional embedding. We distinguish global context  $z_t$ , shared across the sequence, from per-step context  $g_{t,h}$  aligned with the  $h$ -th proposed action. The fused representation and residual are

$$u_h = x_h + \phi_z(z_t) + \phi_g(g_{t,h}), \quad (4)$$

$$\Delta A_t = \psi_{\theta}(\text{Enc}_{\theta}(u_{0:H_p-1})). \quad (5)$$

Here,  $\phi_z$  and  $\phi_g$  are lightweight projections that map global and per-step context into the action-token embedding space,  $\text{Enc}_{\theta}$  is a lightweight temporal sequence encoder, and  $\psi_{\theta}$  is the residual prediction head. For ASRR-A, both context terms are omitted and  $u_h = x_h$ ; ASRR-C includes the context terms specified by its configuration. Global context may contain qpos, low-dimensional observations, task embeddings, or trajectory-mode summaries, whereas per-step context may contain denoising or action-generation hidden features aligned with individual actions. This separation provides a common interface across policy families: the edited object is always the action sequence, while available conditioning depends on what the frozen policy exposes.

When conservative edits are desirable, we optionally bound the residual coordinate-wise as

$$\Delta A_t = \delta_{\max} \odot \tanh(\psi_\theta(\text{Enc}_\theta(u_{0:H_p-1}))), \quad (6)$$

where  $\delta_{\max} \in \mathbb{R}^{d_a}$  bounds each coordinate of the unscaled residual and is shared across timesteps. The element-wise  $\tanh$  enforces these ranges. The mask  $M$  determines which coordinates are edited, while  $\alpha$  additionally scales the final action change in Eq. (2).

### C. Offline Refiner Training

ASRR reuses demonstrations already available for policy adaptation. At each demonstration state, the frozen policy produces  $A_{b,t}$  and any exposed context, while the corresponding future demonstration actions provide the target  $A_t^*$ . These tuples are cached once, after which refiner optimization proceeds entirely offline with the base policy fixed. The cache decouples refiner updates from base-policy inference and backpropagation; no additional environment interaction or reward optimization is needed.

Let  $\nabla_h \Delta a_t = \Delta a_t^h - \Delta a_t^{h-1}$  denote the temporal residual difference. We optimize the composed sequence with

$$\begin{aligned} \mathcal{L}(\theta) = \mathbb{E}_{\mathcal{D}} \left[ \frac{1}{Z_w} \sum_{h=0}^{H_p-1} w_h \ell(\tilde{a}_t^h, a_t^{*,h}) \right. \\ \left. + \lambda_\Delta \|M \odot \Delta A_t\|_F^2 \right. \\ \left. + \lambda_s \sum_{h=1}^{H_p-1} \|M_h \odot \nabla_h \Delta a_t\|_2^2 \right], \end{aligned} \quad (7)$$

Here,  $\mathcal{D}$  denotes the cached training tuples,  $M_h$  is the mask for the  $h$ -th action, and  $\ell$  is a supervised reconstruction loss. The horizon weights  $w_h$  are normalized by  $Z_w = \sum_h w_h$ . The coefficients  $\lambda_\Delta$  and  $\lambda_s$  control residual-magnitude and temporal-smoothness regularization, respectively. Each policy instantiation uses the applicable regularization terms. Proposals, targets, masks, and residuals are represented in the same policy-native action space, preserving the existing action-decoding and conversion pipeline.

### D. Execution-Aligned Refinement

Action-sequence policies may predict more future actions than they directly use before replanning. Let  $H_p$  denote the predicted sequence length and  $H_r \leq H_p$  the portion editable by ASRR; the timestep component of  $M$  leaves actions beyond  $H_r$  unchanged. The refined sequence is then passed to the native execution operator of the frozen policy,

$$A_t^{\text{exec}} = \mathcal{E}_{\text{base}}(\tilde{A}_t), \quad (8)$$

where  $\mathcal{E}_{\text{base}}$  may perform direct prefix execution or temporal aggregation. ASRR edits the proposal while preserving the controller's native replanning rule.

The useful  $H_r$  depends on how much of the proposal influences execution. ACT exposes this sensitivity because direct chunk execution and temporal aggregation use the predicted sequence differently; Section IV-B evaluates this effect. For DP and VLA policies, we retain the native action horizon and execution protocol of the base controller.

## IV. EXPERIMENTS

We organize the experiments around four questions. First, can an established action-sequence policy be improved by editing its proposal alone? Second, does the same output-space interface extend to modern VLA policies? Third, how do base competence, policy-side context, and adaptation cost shape the value of refinement? Finally, do these gains transfer to physical robot execution? ACT and DP, VLA comparisons, and three real-robot tasks address these questions in turn.

### A. Experimental Setup

We evaluate ACT on bimanual insertion and transfer cube, DP on Robomimic Square and Transport [14], and  $\pi_{0.5}$  and OpenVLA-OFT on all four LIBERO suites [25]. The  $\pi_{0.5}$  policy is adapted to LIBERO with LoRA, whereas OpenVLA-OFT uses the released LIBERO fine-tuned checkpoint. All base policies remain frozen during ASRR training.

All refiners are trained offline from cached base proposals and demonstration targets in the corresponding policy-native action space. ACT evaluates action-only and qpos-conditioned refinement, DP evaluates action and low-dimensional context variants, and VLA refiners use bounded, masked corrections while preserving native execution protocols. For  $\pi_{0.5}$ ,  $\alpha$  is selected by paired success on rollout states separate from the final test, with a candidate calibrated on held-out demonstrations included in the search. Each shared refiner uses one scale across all suites.

Success rate is the primary metric, and Table I reports the evaluation count  $N$ . The  $\pi_{0.5}$  comparisons use 400 paired rollouts per suite, with aggregate gain intervals estimated by paired bootstrap resampling within tasks. OpenVLA-OFT results are averaged over three 500-pair rounds. Rescue denotes a base failure corrected by ASRR, whereas harm denotes a base success changed to failure; these counts are totals except for OpenVLA-OFT, which reports per-round means.

### B. Refining Action-Sequence Policies

Table I summarizes the simulation results. ACT and DP isolate what can be learned from the proposal and when context helps.

ACT provides the cleanest test of whether an action proposal is itself repairable. The action-only refiner improves temporal insertion success by 21.7 pp and transfer cube by 6.0 pp, without visual or state input to the refiner. Providing qpos as additional context does not improve over action-only refinement in the main temporal setting. These results show that the predicted action chunk itself provides useful information for local correction.

Execution mode changes the useful refinement horizon (Fig. 3): direct chunk execution favors H30, whereas temporal aggregation favors H80. The editable horizon thus depends on how the controller consumes the sequence.

DP extends the interface from deterministic chunks to denoised trajectories. The selected configurations improve Square by 10.4 pp and Transport by 8.0 pp on average. Table I selects across configurations for each task/checkpoint,

TABLE I

SIMULATION RESULTS.  $N$  DENOTES EVALUATIONS PER CONDITION;  $\Delta$  IS PERCENTAGE-POINT GAIN. PARAMETERS ARE BASE/REFINER MILLIONS; DP REPORTS ITS STANDARD CONTEXT-CONDITIONED REFINER. EVALUATION PROTOCOLS AND METRIC DEFINITIONS APPEAR IN SECTION IV-A.

| Family      | Method             | Task / suite ( $N$ )         | Base (%) | Refined (%) | $\Delta$ (pp) | Rescue | Harm | Params (M)  |
|-------------|--------------------|------------------------------|----------|-------------|---------------|--------|------|-------------|
| Policy      | ACT                | Insertion (300)              | 34.0     | 55.7        | +21.7         | +90    | -25  | 83.92/0.160 |
|             |                    | Transfer (300)               | 89.3     | 95.3        | +6.0          | +30    | -12  | 83.92/0.160 |
|             | DP                 | Square (250)                 | 81.6     | 92.0        | +10.4         | +38    | -12  | 65.88/0.682 |
|             |                    | Transport (250)              | 76.0     | 84.0        | +8.0          | +39    | -19  | 66.92/0.709 |
| $\pi_{0.5}$ |                    | Spatial (400)                | 89.5     | 95.8        | +6.3          | +34    | -9   | 3300/4.0    |
|             |                    | Object (400)                 | 75.8     | 93.5        | +17.8         | +83    | -12  | 3300/4.0    |
|             |                    | Goal (400)                   | 76.0     | 85.5        | +9.5          | +53    | -15  | 3300/4.0    |
|             |                    | LIBERO-10 (400)              | 57.3     | 81.5        | +24.3         | +111   | -14  | 3300/4.0    |
| VLA         | $\pi_{0.5}$ shared | Spatial (400)                | 89.5     | 96.0        | +6.5          | +34    | -8   | 3300/4.0    |
|             |                    | Object (400)                 | 75.8     | 92.3        | +16.5         | +80    | -14  | 3300/4.0    |
|             |                    | Goal (400)                   | 76.0     | 84.8        | +8.8          | +51    | -16  | 3300/4.0    |
|             |                    | LIBERO-10 (400)              | 57.3     | 80.3        | +23.0         | +112   | -20  | 3300/4.0    |
| OpenVLA-OFT |                    | Spatial ( $3 \times 500$ )   | 92.8     | 93.4        | +0.6          | +8.7   | -5.7 | 7540/7.66   |
|             |                    | Object ( $3 \times 500$ )    | 98.8     | 99.1        | +0.3          | +5.0   | -3.3 | 7540/7.66   |
|             |                    | Goal ( $3 \times 500$ )      | 96.4     | 97.3        | +0.9          | +9.3   | -5.0 | 7540/7.66   |
|             |                    | LIBERO-10 ( $3 \times 500$ ) | 95.2     | 96.0        | +0.8          | +11.0  | -7.0 | 7540/7.66   |

while Fig. 4(b) compares results selected within each context type. Observation and observation-plus-mode conditioning achieve the strongest context-specific averages, suggesting that state information helps interpret the denoised proposal. Gains remain positive on average under fresh evaluation seeds, although their magnitude varies with evaluation conditions.

### C. VLA Refinement and Analysis

*a) VLA refinement across tasks:* We next evaluate output-space refinement on VLA policies. At the 10k  $\pi_{0.5}$  checkpoint, suite-specific ASRR-C improves all four LIBERO suites, with a 14.4 pp average gain (Table I). The

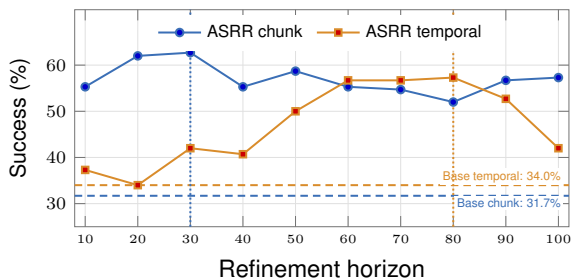


Fig. 3. Effect of refinement horizon on ACT insertion. Curves use three seeds; dashed base references use six seeds. Direct chunk execution favors H30 and temporal aggregation H80.

gains extend action-sequence editing to native flow-generated proposals while keeping the 3.3B-parameter base frozen.

A jointly trained ASRR-C refiner achieves a similar 13.7 pp average gain and improves every suite using a single model. Its performance remains close to suite-specific refinement, showing that a common output-space refiner can serve related task suites.

OpenVLA-OFT provides an additional evaluation on a high-success public base. ASRR yields positive mean changes on all four suites, ranging from 0.3 to 0.9 pp. The following same-run study examines how refinement changes with training progress within one policy architecture.

*b) When refinement is most useful:* We compare checkpoints from the same  $\pi_{0.5}$  LoRA run with their corresponding refiners (Fig. 4(a)). Average base success increases from 62.0% at 5k to 91.3% at 30k, while the average suite-specific gain decreases from 18.9 to 4.6 pp. Shared refinement exhibits the same overall tendency, with gains of 18.3 and 4.3 pp at these endpoints. All aggregate paired 95% gain intervals lie above zero. Across the evaluated checkpoints, refinement provides larger absolute improvements when the base already produces useful behavior but still makes frequent errors, while remaining beneficial at higher base success rates.

*c) Context and adaptation efficiency:* Following the ACT and DP comparisons, we examine action-only and context-conditioned refinement on  $\pi_{0.5}$  (Fig. 4(b)). At 10k,

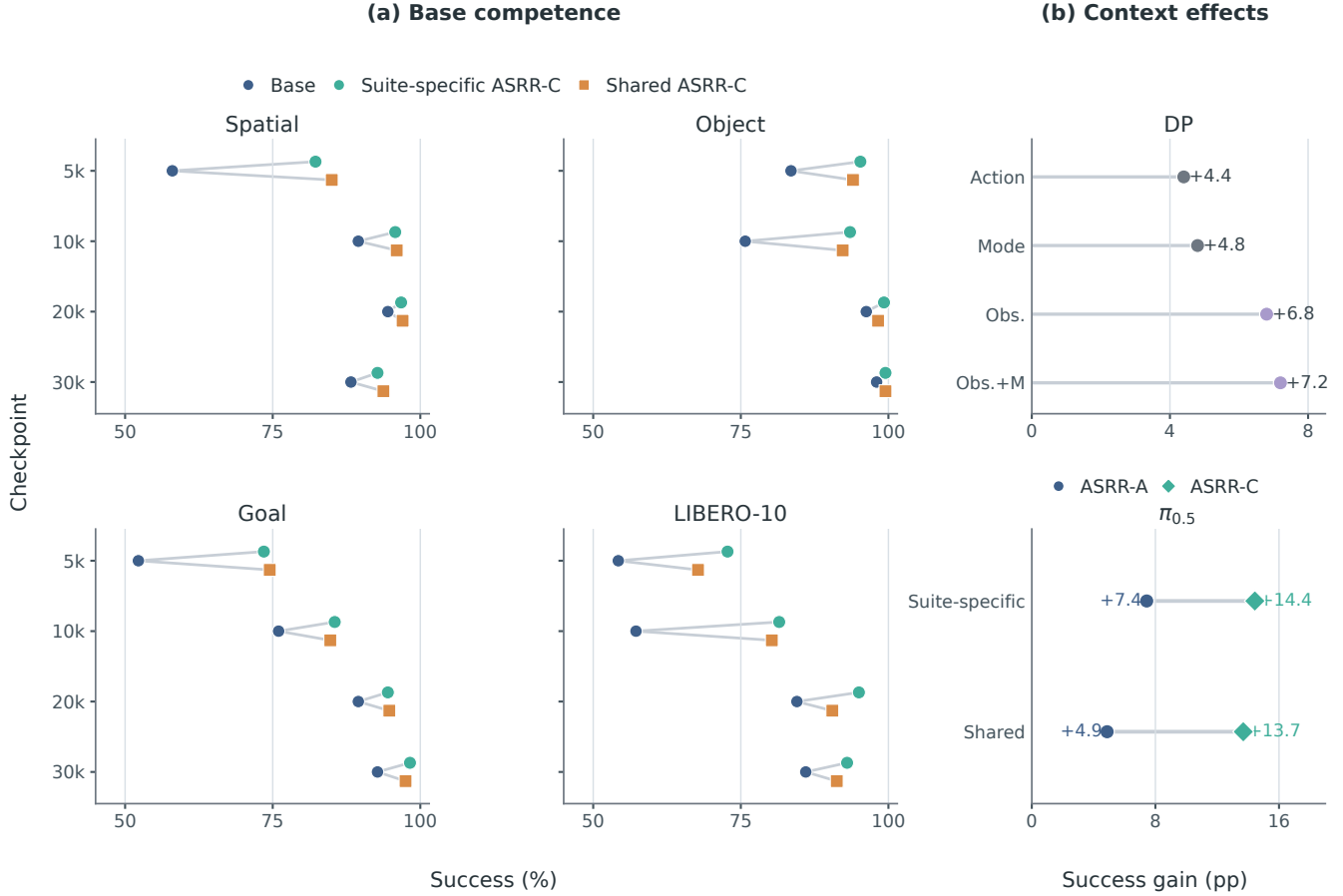


Fig. 4. Refinement across policy checkpoints and context configurations. (a) Same-run  $\pi_{0.5}$  results over 400 paired rollouts per suite, comparing Base, suite-specific ASRR-C, and shared ASRR-C. (b) DP context variants (top) and the 10k  $\pi_{0.5}$  ASRR-A/ASRR-C comparison (bottom).

TABLE II  
RECORDED COMPUTE FOR CONTINUED LoRA AND ASRR.

| Checkpoint pair        | 5k→10k             | 10k→20k            | 20k→30k            |
|------------------------|--------------------|--------------------|--------------------|
| Continued LoRA (GPU-h) | 15.79              | 31.51              | 31.18              |
| ASRR cache (GPU-h)     | 11.62              | 11.42              | 11.40              |
| ASRR fitting (GPU-h)   | 0.19               | 0.13               | 0.12               |
| Compute reduction (%)  | <b>25.2</b>        | <b>63.3</b>        | <b>63.1</b>        |
| Success (%)            | 62.0 → 80.9 / 74.6 | 74.6 → 89.1 / 91.2 | 91.2 → 96.4 / 91.3 |
| ASRR – later Base (pp) | +6.3               | −2.1               | +5.1               |

Four-suite results; ASRR-C keeps the earlier checkpoint frozen. Success: starting Base → ASRR / later Base. GPU-hours sum continued LoRA optimization or recorded ASRR cache extraction and fitting; rollout-based scale selection and final evaluation are excluded.

the average suite-specific gain increases from 7.4 pp with ASRR-A to 14.4 pp with ASRR-C; the corresponding shared-refiner gains are 4.9 and 13.7 pp. Context-conditioned refinement therefore achieves higher success in both configurations. The variants also differ in parameter count, so the comparison reflects their combined input and capacity differences.

*Adaptation cost.* Table II compares suite-specific ASRR-C with continued LoRA from the same checkpoint. Both use

RTX 5880 Ada GPUs: four for LoRA and one per cache or refiner job. Recorded cache extraction and refiner fitting use 25–63% fewer GPU-hours than continued LoRA. ASRR exceeds the later base’s mean success in two comparisons and falls 2.1 pp short in the remaining one.

Each  $\pi_{0.5}$  refiner has approximately 4M parameters, 0.12% of its base, and fits in a few minutes after caching. Cache extraction dominates the recorded cost, but its outputs can be reused across refiner variants at the same checkpoint and demonstration set. This separates repeated corrective optimization from expensive policy inference.

#### D. Real-Robot Evaluation

We evaluate corn-to-plate, block-to-holder, and block-stacking tasks using a Piper arm and two Intel RealSense D455 cameras. Within each task, both methods use the same frozen base policy and are evaluated over 80 trials each with manual resets.

ASRR improves all three tasks (Fig. 5). Corn-to-plate rises from 58/80 to 65/80 successes (+8.8 pp), block-to-holder from 53/80 to 57/80 (+5.0 pp), and block stacking from 43/80 to 48/80 (+6.3 pp). Aggregated across 240 trials per method, success increases from 154/240 (64.2%) to 170/240 (70.8%), a 6.7 pp gain.

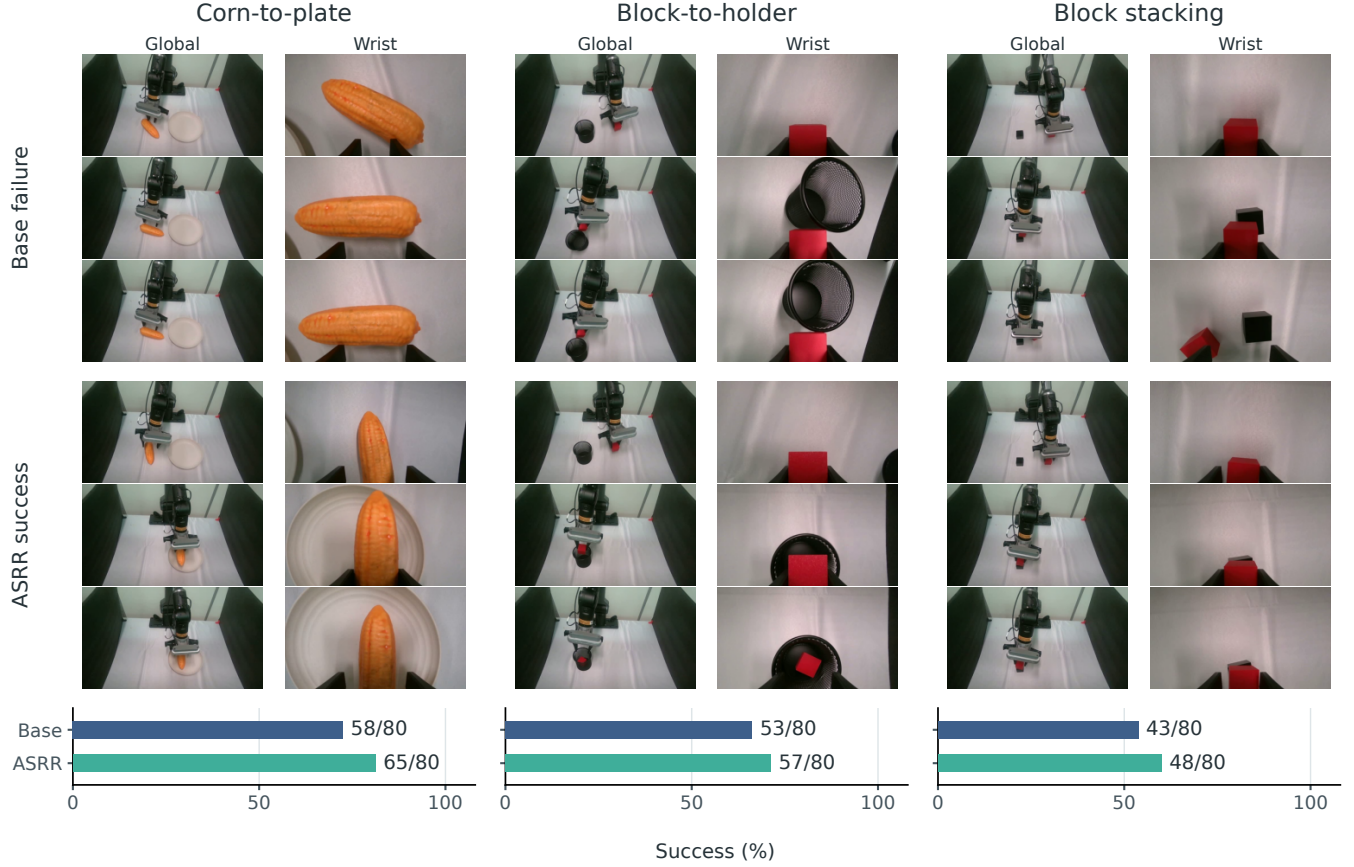


Fig. 5. Selected Base failures and ASRR successes on three real-robot tasks; examples are not paired trials. Global and wrist views occupy the left and right columns of each sequence, with stages progressing downward. Bars report all 80 trials per method and task.

## V. DISCUSSION

*a) Action proposals as adaptation interfaces:* The ACT results suggest that the temporal structure expressed by a policy can support correction without reprocessing perception or updating base weights. DP and the VLA instances extend this view: context can help interpret the proposal, while the object being edited remains the action sequence. A shared interface does not require identical feature encoders or one set of weights across unrelated policies.

*b) Intended regime and intervention strength:* The same-run  $\pi_{0.5}$  study shows larger absolute gains at earlier checkpoints. The compute comparison supports concentrating further optimization in a refiner once meaningful task behavior has emerged. Rescue/harm outcomes also show that an aggregate gain does not imply that every individual correction is beneficial.

*c) Scope and limitations:* Static demonstration caches may not cover the states visited after repeated correction, and pose-only refinement leaves gripper commands unchanged. DP gains weaken under fresh evaluation seeds, while the  $\pi_{0.5}$  study covers one base-training run and one formal evaluation round. The physical study covers three tasks on one robot setup. Bounding the residual limits action changes; collision

avoidance still requires separate assessment. Broader deployment therefore requires evaluating the refined controller under its intended operating conditions.

## VI. CONCLUSION

We proposed and validated direct editing of predicted action sequences as an adaptation interface for competent but imperfect robot policies. ASRR implements this idea with a lightweight residual refiner trained offline on cached policy outputs and existing demonstrations, while keeping the base policy fixed. Its training procedure is straightforward: demonstration actions provide supervised targets, and optimization involves only the small refiner. Reusable caches support repeated refiner optimization, while refined proposals follow the original controller’s execution rule. Experiments across conventional action-sequence policies, VLAs, and three real-robot manipulation tasks demonstrate the practical value of this design. Checkpoint comparisons further show that refinement can approach or exceed later base performance with lower recorded cache-and-fitting compute than continued LoRA optimization. Separating action generation from correction thus offers a practical way to focus a limited adaptation budget on improving an already useful policy.

## ACKNOWLEDGMENT

OpenAI ChatGPT and Codex assisted with experiment and plotting code, manuscript drafting, and language editing in the Abstract and Sections I–VI. The authors reviewed and revised all AI-assisted material and validated the implementation and reported results.

## REFERENCES

- [1] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.
- [2] N. M. Shafiullah, Z. Cui, A. Altanzaya, and L. Pinto, “Behavior transformers: Cloning  $k$  modes with one stone,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 22 955–22 968. [Online]. Available: [https://proceedings.neurips.cc/paper\\_files/paper/2022/file/90d17e882adbdda42349db6f50123817-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2022/file/90d17e882adbdda42349db6f50123817-Paper-Conference.pdf)
- [3] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10–11, pp. 1684–1704, 2025.
- [4] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “RT-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [5] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choro-manski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [6] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [7] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine, “Octo: An open-source generalist robot policy,” *arXiv preprint arXiv:2405.12213*, 2024.
- [8] M. Shukor, D. Aubakirova, F. Capuano, P. Kooyimans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Aliberti, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [9] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “ $\pi_0$ : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [10] K. Black, N. Brown, J. Darpanian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, “ $\pi_{0.5}$ : a vision-language-action model with open-world generalization,” in *9th Annual Conference on Robot Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=vlhoswksBO>
- [11] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [12] A. Agha, K. Otsu, B. Morrell *et al.*, “NeBula: Quest for robotic autonomy in challenging environments; TEAM CoSTAR at the DARPA subterranean challenge,” *arXiv preprint arXiv:2103.11470*, 2021.
- [13] D. A. Robb, J. Scharff Willners, N. Valeyrie, F. J. Chiyah Garcia, A. Laskov, X. Liu, P. Patron, H. Hastie, and Y. R. Petillot, “A natural language interface with relayed acoustic communications for improved command and control of AUVs,” in *2018 IEEE/OES Autonomous Underwater Vehicle Workshop (AUV)*, 2018, pp. 1–6.
- [14] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164, 2022, pp. 1678–1690. [Online]. Available: <https://proceedings.mlr.press/v164/mandlekar22a.html>
- [15] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 6023–6029.
- [16] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 2790–2799. [Online]. Available: <https://proceedings.mlr.press/v97/houlsby19a.html>
- [17] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKEeFYf9>
- [18] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, G. Gordon, D. Dunson, and M. Dudík, Eds., vol. 15. Fort Lauderdale, FL, USA: PMLR, 11–13 Apr 2011, pp. 627–635. [Online]. Available: <https://proceedings.mlr.press/v15/ross11a.html>
- [19] T. Silver, K. Allen, J. B. Tenenbaum, and L. P. Kaelbling, “Residual policy learning,” *arXiv preprint arXiv:1812.06298*, 2018.
- [20] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, “ConRFT: A reinforced fine-tuning method for VLA models via consistency policy,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [21] D. Huang, Z. Fang, T. Zhang, Y. Li, L. Zhao, and C. Xia, “CO-RFT: Efficient fine-tuning of vision-language-action models through chunked offline reinforcement learning,” *arXiv preprint arXiv:2508.02219*, 2025.
- [22] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal, “From imitation to refinement—residual RL for precise assembly,” *arXiv preprint arXiv:2407.16677*, 2024.
- [23] K. Kim, N. Saito, H. Kim, K. Ikeuchi, J. Choo, and Y. Matsushita, “Object-centric residual RL for zero-shot sim-to-real VLA enhancement,” *arXiv preprint arXiv:2606.18953*, 2026.
- [24] E. Welte, Y. Shi, R. Wolf, M. Gilles, and R. Rayyes, “FlowCorrect: Efficient interactive correction of generative flow policies for robotic manipulation,” *arXiv preprint arXiv:2602.22056*, 2026.
- [25] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 44 776–44 791. [Online]. Available: [https://proceedings.neurips.cc/paper\\_files/paper/2023/file/8c3c666820ea055a77726d66fc7d447f-Paper-Datasets\\_and\\_Benchmarks.pdf](https://proceedings.neurips.cc/paper_files/paper/2023/file/8c3c666820ea055a77726d66fc7d447f-Paper-Datasets_and_Benchmarks.pdf)